

MODUL 7

STACK

A. TUJUAN PRAKTIKUM

- Mampu memahami konsep stack pada struktur data dan algoritma
- Mampu mengimplementasikan operasi-operasi pada stack
- Mampu memecahkan permasalahan dengan solusi stack

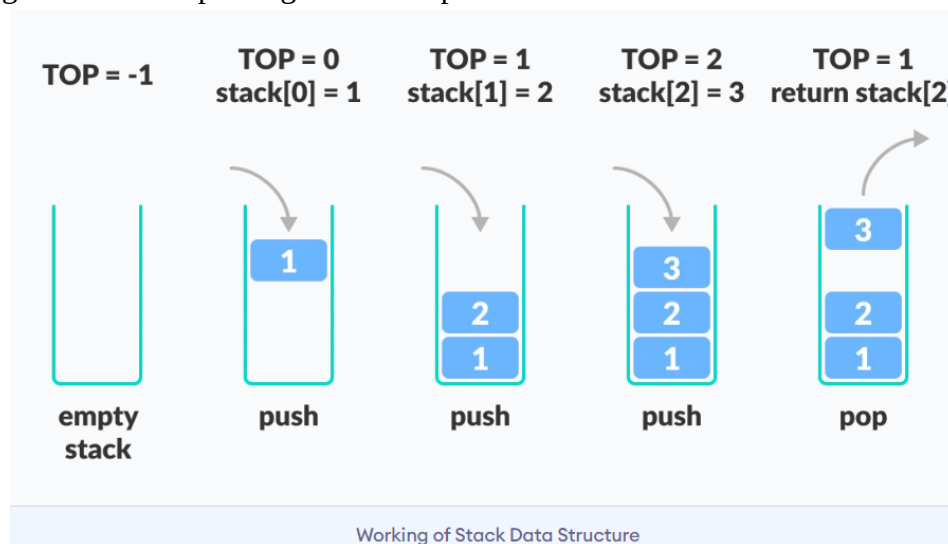
B. DASAR TEORI

Sebuah stack atau tumpukan merupakan struktur data yang berfungsi untuk menyimpan dan mengelola kumpulan data dengan prinsip Last-In, First-Out (LIFO). Analogi yang sering digunakan adalah tumpukan piring di kafetaria, di mana piring terakhir yang ditambahkan akan menjadi yang pertama diambil.

Dalam implementasinya, stack dapat direpresentasikan sebagai struktur data terurut yang memiliki dua operasi utama: push dan pop. Operasi push digunakan untuk menambahkan elemen baru ke dalam stack, sementara operasi pop digunakan untuk menghapus elemen teratas dari stack.

Prinsip LIFO yang menjadi dasar stack membuatnya sangat bermanfaat dalam berbagai aplikasi, termasuk manajemen memori komputer, evaluasi ekspresi aritmatika, dan manajemen panggilan fungsi dalam pemrograman. Sebagai contoh, dalam manajemen memori, stack digunakan untuk menyimpan alamat-alamat memori yang dialokasikan untuk variabel dan fungsi.

Dengan prinsip LIFO ini, stack memungkinkan akses data dengan efisiensi, di mana elemen yang terakhir dimasukkan akan menjadi yang pertama diambil. Hal ini menjadikannya salah satu struktur data yang sangat penting dalam pengembangan perangkat lunak dan pemrograman komputer..



Operasi pada stack melibatkan beberapa fungsi dasar yang dapat dilakukan pada struktur data ini. Berikut adalah beberapa operasi umum pada stack:

- a. Push (Masukkan): Menambahkan elemen ke dalam tumpukan pada posisi paling atas atau ujung.
- b. Pop (Keluarkan): Menghapus elemen dari posisi paling atas atau ujung tumpukan.
- c. Top (Atas): Mendapatkan nilai atau melihat elemen teratas pada tumpukan tanpa menghapusnya.
- d. IsEmpty (Kosong): Memeriksa apakah tumpukan kosong atau tidak.
- e. IsFull (Penuh): Memeriksa apakah tumpukan penuh atau tidak (terutama pada implementasi tumpukan dengan kapasitas terbatas).
- f. Size (Ukuran): Mengembalikan jumlah elemen yang ada dalam tumpukan.
- g. Peek (Lihat): Melihat nilai atau elemen pada posisi tertentu dalam tumpukan tanpa menghapusnya.
- h. Clear (Hapus Semua): Mengosongkan atau menghapus semua elemen dari tumpukan.
- i. Search (Cari): Mencari keberadaan elemen tertentu dalam tumpukan.

C. GUIDED

Sourcecode

```
#include <iostream>
using namespace std;

string arrayBuku[5];
int maksimal = 5, top = 0;

bool isFull() {
    return (top == maksimal);
}

bool isEmpty() {
    return (top == 0);
}

void pushArrayBuku(string data) {
    if (isFull()) {
        cout << "Data telah penuh" << endl;
    } else {
        arrayBuku[top] = data;
        top++;
    }
}

void popArrayBuku() {
    if (isEmpty()) {
        cout << "Tidak ada data yang dihapus" << endl;
    } else {
        arrayBuku[top - 1] = "";
        top--;
    }
}
```

```

void peekArrayBuku(int posisi) {
    if (isEmpty()) {
        cout << "Tidak ada data yang bisa dilihat" << endl;
    } else {
        int index = top;
        for (int i = 1; i <= posisi; i++) {
            index--;
        }
        cout << "Posisi ke " << posisi << " adalah " <<
arrayBuku[index] << endl;
    }
}

int countStack() {
    return top;
}

void changeArrayBuku(int posisi, string data) {
    if (posisi > top) {
        cout << "Posisi melebihi data yang ada" << endl;
    } else {
        int index = top;
        for (int i = 1; i <= posisi; i++) {
            index--;
        }
        arrayBuku[index] = data;
    }
}

void destroyArraybuku() {
    for (int i = top; i >= 0; i--) {
        arrayBuku[i] = "";
    }
    top = 0;
}

void cetakArrayBuku() {
    if (isEmpty()) {
        cout << "Tidak ada data yang dicetak" << endl;
    } else {
        for (int i = top - 1; i >= 0; i--) {
            cout << arrayBuku[i] << endl;
        }
    }
}

int main() {
    pushArrayBuku("Kalkulus");
    pushArrayBuku("Struktur Data");
    pushArrayBuku("Matematika Diskrit");
    pushArrayBuku("Dasar Multimedia");
    pushArrayBuku("Inggris");

    cetakArrayBuku();
    cout << "\n";

    cout << "Apakah data stack penuh? " << isFull() << endl;
    cout << "Apakah data stack kosong? " << isEmpty() << endl;
}

```

```

    peekArrayBuku(2);
    popArrayBuku();

    cout << "Banyaknya data = " << countStack() << endl;

    changeArrayBuku(2, "Bahasa Jerman");
    cetakArrayBuku();

    cout << "\n";

    destroyArraybuku();
    cout << "Jumlah data setelah dihapus: " << top << endl;

    cetakArrayBuku();

    return 0;
}

```

Output

```

Inggris
Dasar Multimedia
Matematika Diskrit
Struktur Data
Kalkulus

Apakah data stack penuh? 1
Apakah data stack kosong? 0
Posisi ke 2 adalah Dasar Multimedia
Banyaknya data = 4
Dasar Multimedia
Bahasa Jerman
Struktur Data
Kalkulus

Jumlah data setelah dihapus: 0
Tidak ada data yang dicetak

```

D. UNGUIDED

1. Buatlah program untuk menentukan apakah kalimat tersebut yang diinputkan dalam program stack adalah palindrom/tidak. Palindrom kalimat yang dibaca dari depan dan belakang sama. Jelaskan bagaimana cara kerja programnya.

Contoh:

Kalimat : ini

Kalimat tersebut adalah polindrom

Kalimat : telkom

Kalimat tersebut adalah bukan polindrom

```
Masukan Kalimat : ini
Kalimat tersebut adalah : Palindrom
```

Code:

```
#include <iostream>
#include <stack>
#include <string>
#include <algorithm>

bool isPalindrome(const std::string& str) {
    std::stack<char> charStack;
    std::string cleanedStr;

    // Menghapus spasi dan mengubah ke huruf kecil untuk memeriksa palindrom
    for (char ch : str) {
        if (isalnum(ch)) { // hanya memasukkan huruf dan angka
            cleanedStr += tolower(ch);
        }
    }

    // Masukkan semua karakter ke dalam stack
    for (char ch : cleanedStr) {
        charStack.push(ch);
    }

    // Periksa dengan membandingkan karakter satu per satu dari awal dan akhir
    for (char ch : cleanedStr) {
        if (ch != charStack.top()) {
            return false;
        }
        charStack.pop();
    }
    return true;
}

int main() {
    std::string input;
    std::cout << "Masukan Kalimat: ";
    std::getline(std::cin, input);

    if (isPalindrome(input)) {
        std::cout << "Kalimat tersebut adalah: Palindrom" << std::endl;
    } else {
        std::cout << "Kalimat tersebut adalah: Bukan Palindrom" << std::endl;
    }

    return 0;
}
```

Output:

Program memastikan bahwa input dapat mengabaikan perbedaan besar/kecil serta spasi, sehingga lebih fleksibel dalam menentukan apakah sebuah kalimat adalah palindrom atau bukan.

```
Masukan Kalimat: ini
Kalimat tersebut adalah: Palindrom
```

```
Masukan Kalimat: telkom
Kalimat tersebut adalah: Bukan Palindrom
```

Cara kerja:

- Input & Normalisasi kalimat : Program meminta pengguna agar masukkan kalimat setelah itu hapus karakter selain huruf dan angka yang dikoverensi semua karakter menjadi huruf kecil. Tujuannya agar mengabaikan perbedaan huruf besar/kecil dan spasi dalam pengecekan palindrom.
 - Masukkan karakter dalam Stack: Semua yang sudah dinormalisasi masuk dalam stack, karena sesuai prinsip *Last In, First Out* (LIFO) yaitu urutan karakter pada stack akan terbalik dari aslinya.
 - Mengecek karakter: Mengambil setiap karakter dari awal string yang sudah dinormalisasi lalu bandingkan dengan karakter paling atas dari stack. Jika ada yang beda berarti kalimat bukan palindrom serta fungsi false.
 - Hasil: Jika semua cocok maka program true, berarti kalimat tersebut adalah palindrom.
2. Buatlah program untuk melakukan pembalikan terhadap kalimat menggunakan stack dengan minimal 3 kata. Jelaskan output program dan source codenya beserta operasi/fungsi yang dibuat?

Contoh

Kalimat : Telkom Purwokerto

Hasil : otrekowruP mokleT

```
Masukkan Kata Telkom Purwokerto
Datastack Array :
Data : otrekowruP mokleT
```

Code:

- **`std::stack<char> charStack`** [membuat stack untuk menampung karakter-karakter dari kalimat input]
- **`charStack.push(ch);`** [masukkan karakter di dalam stack]
- **`charStack.pop();`** [hapus karakter paling atas dari stack]
- **`charStack.top();`** [Mengambil karakter paling atas dari stack tanpa menghapusnya]

- `std::getline(std::cin, input);` [baca input kalimat lengkap dari pengguna]

```
#include <iostream>
#include <stack>
#include <string>

void reverseString(const std::string& input) {
    std::stack<char> charStack;

    // Memasukkan semua karakter ke dalam stack
    for (char ch : input) {
        charStack.push(ch);
    }

    // Mengeluarkan karakter dari stack untuk membentuk string yang terbalik
    std::cout << "Data: ";
    while (!charStack.empty()) {
        std::cout << charStack.top();
        charStack.pop();
    }
    std::cout << std::endl;
}

int main() {
    std::string input;
    std::cout << "Masukkan Kata: ";
    std::getline(std::cin, input);

    // Memeriksa apakah input memiliki minimal 3 kata
    int wordCount = 0;
    for (char ch : input) {
        if (ch == ' ') {
            wordCount++;
        }
    }

    if (wordCount >= 2) { // berarti minimal 3 kata karena ada dua spasi
        std::cout << "Datastack Array:" << std::endl;
        reverseString(input);
    } else {
        std::cout << "Masukkan minimal 3 kata!" << std::endl;
    }

    return 0;
}
```

Output:

Jika kalimat minimal 3 kata, maka program akan menampilkan hasil dari pembalikan kalimat, jika tidak maka akan tampil pesan kesalahan.

```
Masukkan Kata: Telkom University Purwokerto
Datastack Array:
Data: otrekowruP ytisrevinU mokleT
```

Cara kerja:

- Input kalimat: pengguna masukkan kalimat terdiri minimal 3 kata, input dibaca dengan `std::getline()` agar menangkap kalimat lengkap
- Periksa jumlah kata: menghitung jumlah spasi pada kalimat agar memastikan kalimat minimal 3 kata. Jika terdapat 2 spasi atau lebih, maka kalimat memenuhi syarat.
- Menggunakan Stack agar membalik kalimat: karakter masuk dalam *stack* (`charStack`). Memiliki sifat *Last In, First Out* (LIFO) jadi karakter yang keluar akan terbalik.

- Cetak kalimat terbalik: setiap program yang keluar dan tampil satu persatu membentuk kalimat terbalik.

E. DAFTAR PUSTAKA

Karumanchi, N. (2016). *Data Structures and algorithms made easy: Concepts, problems, Interview Questions*. CareerMonk Publications.